

# Training a JetBot on Isaac Lab with the Dell Pro Max Powered by NVIDIA RTX PRO 6000 Blackwell

Building a reinforcement learning environment from scratch where a differential-drive robot learns to chase a moving sphere using PPO - trained in minutes on Blackwell-class GPU hardware. From Isaac Lab training to Isaac Sim standalone deployment to a full Isaac Sim extension.

February 2026 | 18 min read

---

Imagine dropping a small wheeled robot into an arena with a glowing green sphere. The robot knows nothing about the world - no maps, no hard-coded rules, no PID controllers. All it has is a neural network brain and a reward signal that says: "get closer to the sphere." Within minutes of simulated training across 100 parallel worlds, the robot figures out how to spin its wheels to chase the target, and when it catches up, the sphere teleports to a new random location and the hunt begins again.

This is the power of reinforcement learning (RL) in simulation. Running on a Dell Pro Max workstation equipped with the NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM), we can spin up 100 parallel physics environments and train a policy in roughly 10 minutes.

This document covers the complete journey in three chapters:

- Chapter I: Isaac Lab - Designing the RL environment, observations, rewards, and training the PPO policy across 100 parallel environments.
- Chapter II: Isaac Sim Standalone - Deploying the trained checkpoint in a standalone Isaac Sim script outside the Isaac Lab framework.
- Chapter III: Isaac Sim Extension - Packaging the deployment as a proper Isaac Sim extension with UI controls, registered in the Examples Browser.

# Chapter I: Isaac Lab - Training the Policy

In this chapter we build the complete training pipeline: defining the robot, the sphere-following task, ego-centric observations, a shaped reward function, and training with PPO using the skrl library.

## 1.1 The Platform: Isaac Lab + JetBot

### What is Isaac Lab?

Isaac Lab is NVIDIA's open-source framework for robot learning built on top of Isaac Sim and the Omniverse platform. It provides GPU-accelerated physics simulation, parallel environment execution, and first-class integration with RL libraries like skrl, rl\_games, RSL-RL, and Stable Baselines3.

The key advantage is scale: Isaac Lab can run hundreds or thousands of environments in parallel on a single GPU. Each environment is a full physics simulation with articulated robots, collision geometry, and contact forces - all running at 120 Hz.

### The JetBot: A Differential-Drive Robot

The NVIDIA JetBot is a two-wheeled differential-drive robot. It has exactly two actuated joints - a left wheel and a right wheel. By varying the velocity of each wheel independently, the robot can move forward, turn left, turn right, spin in place, or move in curved arcs.

This simplicity makes it an ideal testbed for RL:

- Action space: 2 continuous values (left wheel velocity, right wheel velocity)
- Dynamics: nonholonomic - the robot cannot move sideways directly
- Challenge: the agent must learn coordinated wheel commands to steer toward a goal

### Environment Configuration

The environment configuration defines the simulation parameters:

```
@configclass
class SphereFollowEnvCfg(DirectRLEnvCfg):
    decimation = 2                # control at 60 Hz
    episode_length_s = 20.0        # 20-second episodes
    action_space = 2               # left/right wheel velocity
    observation_space = 4          # see Section 1.3
    env_spacing = 4.0              # meters between envs
    sphere_radius = 0.1            # 10cm green sphere
    sphere_reach_threshold = 0.3    # "reached" at 30cm
    sphere_spawn_range_min = 0.5    # spawn 0.5-1.5m away
    sphere_spawn_range_max = 1.5
```

## 1.2 The Task: Sphere Following

The task is conceptually simple: a green sphere spawns at a random position near the robot. The robot must navigate to the sphere. When it gets within 30cm, the sphere teleports to a new random location and the robot must chase it again. This cycle continues for the entire 20-second episode.

The sphere is implemented as a visualization marker - it has no physics collider and no mass. Its position is controlled entirely by the environment code.

Each episode proceeds as follows:

- Robot resets to its default pose at the center of the env cell
- Sphere spawns at a random angle, 0.5-1.5m from the robot
- Robot takes actions (wheel velocities) based on observations
- When robot reaches sphere (dist < 0.3m): +5.0 bonus, sphere respawns
- Episode ends after 20 seconds (timeout)

## 1.3 Observation Space Design

Designing good observations is one of the most critical decisions in RL environment design. Our observation is a 4-dimensional vector computed every step:

```
obs = [dot, cross_z, dist_norm, forward_speed]
```

### 1. Dot Product (alignment signal)

$\text{dot} = \text{sum}(\text{forward\_dir} * \text{dir\_to\_sphere})$ . Equals +1 when the robot faces directly toward the sphere, -1 when facing away. It tells the agent HOW WELL it is aligned with the target.

## 2. Cross Product Z-component (steering signal)

$\text{cross\_z} = (\text{forward} \times \text{dir\_to\_sphere}).z$ . Positive when the sphere is to the left, negative when to the right. Combined with the dot product, these two signals fully encode the relative bearing to the sphere.

## 3. Normalized Distance

$\text{dist\_norm} = \text{distance} / 3.0$ , clamped to  $[0, 1]$ . Tells the agent HOW FAR the sphere is, normalized to a neural-network-friendly range.

## 4. Forward Speed

$\text{forward\_speed} = \text{body-frame X velocity}$ . Gives the agent proprioceptive feedback about its current motion.

*These four numbers are sufficient for the agent to solve the task. The ego-centric representation generalizes better across environments than raw positions or quaternions.*

# 1.4 Reward Function Engineering

Our reward is a sum of four components:

```
reward = approach + alignment + reach_bonus + time_penalty
```

## Approach Reward (dense, distance-based)

```
approach = (prev_dist - curr_dist) * 1.0
```

A potential-based shaping reward. Every step, we compare the current distance to the previous distance. If the robot moved closer, the reward is positive.

## Alignment Reward (orientation incentive)

```
alignment = dot(forward, dir_to_sphere) * 0.5
```

Rewards the robot for facing the sphere, even before it starts moving. Encourages turn-then-drive behavior.

## Reach Bonus (sparse, goal-based)

```
reach = 5.0 if dist < 0.3m else 0.0
```

A one-time bonus when the robot reaches the sphere. After collecting it, the sphere respawns at a new random location.

## Time Penalty (efficiency pressure)

```
time_penalty = -0.01 per step
```

A small constant penalty discouraging the agent from spinning or sitting still.

# 1.5 The Control System

The trained policy is a neural network that maps observations to actions. At each control step (60 Hz), the following pipeline executes:

1. Physics sim advances (120 Hz, decimation=2)
2. Read robot state: position, orientation, velocity
3. Compute 4D observation vector
4. Neural network forward pass: obs -> actions
5. Actions = [left\_wheel\_vel, right\_wheel\_vel]
6. Apply velocity targets to wheel joints
7. Compute reward, check termination

## Network Architecture

```

Input:  4 observations
|
v
Linear(4 -> 64) + ELU
Linear(64 -> 64) + ELU
|           |
v           v
Policy:      Value:
Linear(64->2) Linear(64->1)

```

## 1.6 Training with PPO

| Parameter       | Value    | Purpose                      |
|-----------------|----------|------------------------------|
| Environments    | 100      | Parallel data collection     |
| Network         | [64, 64] | Shared policy-value backbone |
| Rollout length  | 48 steps | Experience per update        |
| Learning rate   | 3e-4     | Adaptive via KL threshold    |
| Mini-batches    | 8        | SGD batches per epoch        |
| Learning epochs | 8        | Passes over rollout data     |
| Entropy coeff   | 0.01     | Exploration bonus            |
| Total timesteps | 24,000   | ~10 min on RTX PRO 6000      |
| Discount factor | 0.99     | Long-horizon returns         |

### Training Progression

Phase 1 - Random exploration: The agent outputs near-random wheel velocities. The approach reward provides gradient signal even during random motion.

Phase 2 - Turning behavior emerges: The agent learns that facing the sphere yields positive alignment reward.

Phase 3 - Drive-and-chase: The agent combines turning with forward motion, reaching spheres more frequently.

Phase 4 - Refined tracking: The agent learns smooth, efficient curved trajectories directly to the target.

## 1.7 Checkpoint and Key Implementation

# Details

During training, skrl saves checkpoints and tracks the best agent:

```
logs/skrl/sphere_follow_direct/  
  2026-02-21_19-46-44_ppo_torch/  
    checkpoints/  
      best_agent.pt      # best by episode return  
    params/  
      env.yaml           # environment config  
      agent.yaml         # agent hyperparameters
```

## What the Checkpoint Contains

- Policy network weights and biases (4 -> 64 -> 64 -> 2)
- Value network weights (shared backbone + value head)
- Running mean/std for observation normalization (RunningStandardScaler)
- Optimizer state (for resuming training)

## Sphere Repositioning Without Reward Spikes

When the robot reaches a sphere, we reposition it. A naive implementation would cause a reward spike. Our solution: reset `prev_dist` immediately:

```
reached_ids = torch.where(curr_dist < 0.3)[0]  
if len(reached_ids) > 0:  
    self._spawn_sphere_positions(reached_ids)  
    new_diff = sphere_pos[reached_ids] - robot_pos[reached_ids]  
    curr_dist[reached_ids] = torch.linalg.norm(new_diff)  
self.prev_dist_to_sphere = curr_dist.clone()
```

## Stale Simulation Data During Reset

After `write_root_state_to_sim()`, robot position data is not updated until the next simulation step. We pass the known reset position directly:

```
robot_reset_pos_xy = default_root_state[:, :2]  
self._spawn_sphere_positions(env_ids,  
    robot_pos_xy=robot_reset_pos_xy)
```

## 1.8 Isaac Lab Launch Commands

```
# Install the package
./launch.sh install

# Verify environment registration
./launch.sh list-envs

# Visual smoke test with random agent
./launch.sh random-agent --num_envs 10

# Train the PPO agent (100 parallel envs)
./launch.sh train --algorithm PPO --num_envs 100

# Evaluate the trained agent
./launch.sh play --checkpoint \
    logs/skrl/sphere_follow_direct/<run>/checkpoints/best_agent.pt

# Quick reference
./launch.sh install      # Install package
./launch.sh list-envs    # List registered envs
./launch.sh train        # Train RL agent
./launch.sh play         # Evaluate trained agent
./launch.sh help         # Show all options
```



# Chapter II: Isaac Sim Standalone - Deploying the Trained Policy

Once we have a trained checkpoint from Isaac Lab, we can deploy it in a standalone Isaac Sim script - no Isaac Lab framework required. This is useful for demos, integration testing, or running the policy on machines that have Isaac Sim but not the full Isaac Lab stack.

## 2.1 From Isaac Lab to Isaac Sim: The API Mapping

Isaac Lab provides high-level abstractions (Articulation, DirectRLEnv, math\_utils) that wrap lower-level Isaac Sim APIs. When moving to standalone Isaac Sim, we need to map each Isaac Lab call to its Isaac Sim equivalent:

| Isaac Lab API                   | Isaac Sim Standalone                          |
|---------------------------------|-----------------------------------------------|
| robot.data.root_pos_w           | jetbot.get_world_pose()[0]                    |
| robot.data.root_link_quat_w     | jetbot.get_world_pose()[1]                    |
| robot.data.root_com_lin_vel_b   | quat_rotate_inverse(q, get_linear_velocity()) |
| math_utils.quat_apply(q, v)     | quat_rotate(q, v)                             |
| robot.set_joint_velocity_target | articulation_view.set_joint_velocity_targets  |
| sim dt=1/120, decimation=2      | physics_dt=1/120, rendering_dt=1/60           |

## 2.2 Standalone Script Architecture

The standalone script (scripts/standalone\_sphere\_follow.py) follows this structure:

- Create SimulationApp (must happen before any Isaac Sim imports)
- Define quaternion utilities (quat\_rotate, quat\_rotate\_inverse) for wxyz convention
- Define PolicyNetwork matching the training architecture exactly
- Load checkpoint: extract policy weights + RunningStandardScaler parameters

- Create World with matching physics/render rates
- Spawn JetBot from USD asset, ground plane, dome light, green sphere
- Main loop: read state, compute obs, normalize, run policy, apply action

## Quaternion Utilities (wxyz Convention)

Both Isaac Lab and Isaac Sim use the wxyz (scalar-first) quaternion convention. We implement `quat_rotate` and `quat_rotate_inverse` using the cross-product formula to match Isaac Lab's `math_utils` exactly:

```
def quat_rotate(quat_wxyz, vec):
    """Rotate vector by quaternion (wxyz)."""
    xyz = quat_wxyz[1:4]
    t = 2.0 * np.cross(xyz, vec)
    return vec + quat_wxyz[0] * t + np.cross(xyz, t)

def quat_rotate_inverse(quat_wxyz, vec):
    """World frame -> body frame."""
    xyz = quat_wxyz[1:4]
    t = 2.0 * np.cross(xyz, vec)
    return vec - quat_wxyz[0] * t + np.cross(xyz, t)
```

## Policy Network (Exact Architecture Match)

The PolicyNetwork must exactly match the skrl training architecture. We extract only the policy-relevant weights from the checkpoint:

```

class PolicyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.net_container = nn.Sequential(
            nn.Linear(4, 64), nn.ELU(),
            nn.Linear(64, 64), nn.ELU(),
        )
        self.policy_layer = nn.Linear(64, 2)

# Load from skrl checkpoint:
state_dict = {
    key: ckpt["policy"][key]
    for key in ["net_container.0.weight", ...]
}
# Also extract observation normalizer:
obs_mean = ckpt["state_preprocessor"]["running_mean"]
obs_std = sqrt(ckpt["state_preprocessor"]["running_variance"] + 1e-8)

```

## Observation Normalization

During training, skrl uses a `RunningStandardScaler` to normalize observations. The checkpoint stores the running mean and variance. At inference time, we must apply the same normalization:

```

obs_normalized = (obs_tensor - obs_mean) / obs_std

```

Without this normalization step, the policy outputs would be meaningless - the network was trained on normalized inputs.

## 2.3 The Main Loop

The standalone main loop mirrors the training environment's step logic:

```

while simulation_app.is_running():
    my_world.step(render=True)

    if my_world.is_playing():
        # Read robot state
        robot_pos, robot_quat = jetbot.get_world_pose()
        robot_lin_vel_w = jetbot.get_linear_velocity()

        # Compute 4D observation
        obs = compute_observations(
            robot_pos, robot_quat, robot_lin_vel_w,
            sphere_pos, max_distance=3.0)

        # Normalize + policy forward pass
        obs_norm = (torch.from_numpy(obs) - obs_mean) / obs_std
        action = policy(obs_norm).squeeze().numpy()

        # Apply wheel velocity targets
        vel_targets = np.zeros(jetbot.num_dof)
        vel_targets[left_idx] = action[0]
        vel_targets[right_idx] = action[1]
        jetbot._articulation_view.set_joint_velocity_targets(
            vel_targets.reshape(1, -1))

        # Check sphere reached -> reposition
        if np.linalg.norm(sphere_pos[:2] - robot_pos[:2]) < 0.3:
            sphere_pos = random_sphere_position(robot_pos)
            set_sphere_position(sphere_prim, sphere_pos)

```

## 2.4 Running the Standalone Script

```

python scripts/standalone_sphere_follow.py \
    --checkpoint logs/skrl/sphere_follow_direct/\
    <run>/checkpoints/best_agent.pt

```

The script opens an Isaac Sim window with the JetBot, green sphere, and ground plane. The trained policy runs in real-time - the JetBot actively tracks and reaches the sphere, which repositions after each reach.

Key differences from Isaac Lab evaluation:

- Single environment only (no parallel envs)
- Uses Isaac Sim WheeledRobot API instead of Isaac Lab Articulation

- Green sphere is a USD prim, not a VisualizationMarker
- No episode timeout - runs continuously until the window is closed

# Chapter III: Isaac Sim Extension - The Examples Browser

The final step is packaging the policy deployment as a proper Isaac Sim extension that appears in the Examples Browser alongside NVIDIA's built-in examples (Hello World, Franka, Quadruped, etc.). This provides a polished UI with Load/Reset buttons, checkpoint configuration, and live status display.

## 3.1 Extension Architecture

Isaac Sim extensions follow a well-defined architecture based on the Omniverse Kit framework. The key components are:

- BaseSample: Abstract base class managing the World lifecycle (load, reset, clear). Provides hooks: `setup_scene()`, `setup_post_load()`, `on_physics_step()`, etc.
- BaseSampleUITemplate: UI framework with CollapsibleFrames, Load/Reset buttons, and abstract methods for custom UI controls.
- Extension (`omni.ext.IExt`): Entry point that creates the sample + UI and registers with the Examples Browser.

Our extension consists of three files:

```
sphere_follow/  
  __init__.py          # Exports  
  sphere_follow.py      # SphereFollow(BaseSample)  
  sphere_follow_extension.py  # SphereFollowUI + SphereFollowExtension
```

## 3.2 SphereFollow(BaseSample) - The Scene and Logic

The SphereFollow class inherits from BaseSample and implements all lifecycle methods.

The key pattern: scene setup is synchronous, post-load is async, and the policy runs in a physics callback.

## Scene Setup

```
class SphereFollow(BaseSample):
    def __init__(self):
        super().__init__()
        # Match training: 120Hz physics, 60Hz render
        self._world_settings["physics_dt"] = 1.0 / 120.0
        self._world_settings["rendering_dt"] = 1.0 / 60.0

    def setup_scene(self):
        world = self.get_world()
        world.scene.add_default_ground_plane()

        # Spawn JetBot from USD asset
        self._jetbot = world.scene.add(WheeledRobot(
            prim_path="/World/Jetbot",
            wheel_dof_names=["left_wheel_joint",
                            "right_wheel_joint"],
            usd_path=jetbot_asset_path,
            position=np.array([0, 0, 0.05]),
        ))

        # Green sphere target (USD prim)
        self._sphere_prim = create_green_sphere(stage, ...)

        # Dome light matching training environment
        UsdLux.DomeLight.Define(stage, "/World/DomeLight")
```

## Post-Load: Policy Loading and Physics Callback

```
async def setup_post_load(self):
    # Load trained PPO checkpoint
    self._load_policy()

    # Register physics callback (runs at 120Hz)
    world.add_physics_callback("sphere_follow_step",
                               callback_fn=self.on_physics_step)

    # Start simulation immediately
    await self.get_world().play_async()
```

## Physics Step: Policy Inference at 60Hz

```
def on_physics_step(self, step_size):
    # Decimation: policy every 2 physics steps
    self._decimation_counter += 1
    if self._decimation_counter % 2 != 0:
        return

    # Read state -> compute obs -> normalize -> policy -> apply
    robot_pos, robot_quat = self._jetbot.get_world_pose()
    obs = compute_observations(robot_pos, robot_quat,
                              self._jetbot.get_linear_velocity(), self._sphere_pos)
    obs_norm = (torch.from_numpy(obs) - self._obs_mean) / self._obs_std

    with torch.no_grad():
        action = self._policy(obs_norm).squeeze().numpy()

    # Apply wheel velocity targets
    self._jetbot._articulation_view.set_joint_velocity_targets(...)

    # Check sphere reached -> reposition
    if dist < 0.3:
        self._sphere_pos = random_sphere_position(robot_pos)
        set_sphere_position(self._sphere_prim, self._sphere_pos)
```

## 3.3 The Extension UI

The SphereFollowUI class extends BaseSampleUITemplate to add two extra frames:

- Policy Configuration: A text field for the checkpoint path. The user can change this before clicking Load to point to a different checkpoint.
- Live Status: Real-time display of spheres reached, step count, 4D observations, and wheel velocity actions updated every ~0.5 seconds.



```

class SphereFollowUI(BaseSampleUITemplate):
    def build_extra_frames(self):
        with self.extra_stacks:
            # Checkpoint path field
            checkpoint_frame = ui.CollapsibleFrame(
                title="Policy Configuration", ...)
            with checkpoint_frame:
                self._checkpoint_field = ui.StringField()

            # Live status display
            status_frame = ui.CollapsibleFrame(
                title="Live Status", ...)
            with status_frame:
                self._stats_label = ui.Label("Spheres: 0")
                self._obs_label = ui.Label("Obs: [...]")

```

## Extension Registration

The SphereFollowExtension registers with the Examples Browser so it appears in the Isaac Sim UI under the 'Policy' category:

```

class SphereFollowExtension(omni.ext.IExt):
    def on_startup(self, ext_id):
        sample = SphereFollow()
        ui_handle = SphereFollowUI(
            title="Wheeled Robot: JetBot Sphere Follow",
            sample=sample, ...)

        get_browser_instance().register_example(
            name="JetBot Sphere Follow",
            execute_entrypoint=ui_handle.build_window,
            ui_hook=ui_handle.build_ui,
            category="Policy",
        )

```

## 3.4 Installation and Registration

To install the extension into Isaac Sim, copy the sphere\_follow directory into the interactive examples extension and register it in extension.toml:

```
# File location:
isaacsim/exts/isaacsim.examples.interactive/
  isaacsim/examples/interactive/sphere_follow/
    __init__.py
    sphere_follow.py
    sphere_follow_extension.py

# Add to extension.toml:
[[python.module]]
name = "isaacsim.examples.interactive.sphere_follow"
```

After installation, the extension appears in Isaac Sim's Examples Browser. Navigate to Policy > JetBot Sphere Follow, click Load, and the trained JetBot will begin chasing the green sphere with the standard Load/Reset controls.

## 3.5 Comparing the Three Approaches

|               | Isaac Lab           | Standalone     | Extension        |
|---------------|---------------------|----------------|------------------|
| Use case      | Training + eval     | Quick demo     | Integrated UI    |
| Framework     | Isaac Lab           | Isaac Sim only | Isaac Sim + Kit  |
| Parallel envs | Yes (100+)          | No (single)    | No (single)      |
| UI            | Terminal            | Viewport only  | Full UI panel    |
| Base class    | DirectRLEnv         | None (script)  | BaseSample       |
| Physics loop  | Env step()          | world.step()   | Physics callback |
| Sphere        | VisualizationMarker | USD prim       | USD prim         |
| Robot API     | Articulation        | WheeledRobot   | WheeledRobot     |

---

# Conclusion

We built a complete reinforcement learning pipeline for teaching a JetBot to chase a sphere, then deployed the trained policy in three different ways:

- Isaac Lab: GPU-accelerated parallel training with 100 environments, training a working policy in ~10 minutes on the RTX PRO 6000 Blackwell.
- Isaac Sim Standalone: A self-contained script that loads the checkpoint and runs inference using only Isaac Sim APIs - no Isaac Lab required.
- Isaac Sim Extension: A fully integrated extension with UI controls, registered in the Examples Browser alongside NVIDIA's built-in samples.

Key takeaways:

- Environment design matters: compact observations (4D), shaped rewards (4 components), and proper episode structure are more important than network size or hyperparameter tuning.
- Parallel simulation is transformative: 100 environments on one GPU provide enough data to train a working policy in about 10 minutes.
- The Isaac Lab -> Isaac Sim pipeline is clean: the API mapping is straightforward, and the checkpoint contains everything needed for deployment (weights + observation normalizer).
- The BaseSample/BaseSampleUITemplate pattern makes it easy to create professional Isaac Sim extensions with proper lifecycle management.

---

## Resources

- NVIDIA Isaac Lab: [github.com/isaac-sim/IsaacLab](https://github.com/isaac-sim/IsaacLab)
- NVIDIA Isaac Sim: [docs.isaacsim.omniverse.nvidia.com](https://docs.isaacsim.omniverse.nvidia.com)
- skrl RL Library: [skrl.readthedocs.io](https://skrl.readthedocs.io)
- PPO Paper: Schulman et al., 2017 - [arxiv.org/abs/1707.06347](https://arxiv.org/abs/1707.06347)
- Project Repository: [github.com/kabilankb/isaaclab\\_jetbot](https://github.com/kabilankb/isaaclab_jetbot)

*Built with Isaac Lab 0.47, Isaac Sim 5.1, skrl 1.4, and PyTorch. Trained on Dell Pro Max with NVIDIA RTX PRO 6000 Blackwell (96 GB).*